The Complete Version of IP Protocol

Chang Qing
Beijing IpPlusPlus Network Technology Co., Ltd.,Beijing 100195,China
Email:cq@ippp.xyz

Abstract

IPv4 adopts 32-bit fixed-length addresses, the problem of insufficient address space has long been prominent. As a temporary solution, Network Address Translation (NAT) alleviates address scarcity to a certain extent, yet it breaks the end-to-end communication principle of the Internet. It only allows intranet devices to actively access the extranet and fails to support direct access from the extranet to the intranet. Although various solutions have been proposed in the industry, they all have inherent limitations. This paper presents an innovative scheme named IP++ based on multi-level variable-length addresses, which fundamentally solves the above problems and restores the IP protocol to the basic end-to-end principle. The multi-level variable-length address structure of IP++ is highly compatible with the natural hierarchical topology of the Internet, featuring strong backward compatibility, convenient deployment and excellent usability.

Keywords: variable-length address; end-to-end;IP++; encapsulation; translation

1 Background of the Problem

The network technology system has evolved gradually rather than being fully designed at one time [1, 2].Traditional IPv4 uses 32-bit addresses [3]. With the continuous expansion of the Internet scale, the shortage of address resources has become increasingly severe. To address this issue, Network Address Translation (NAT) was introduced [4]. Essentially, NAT enables multiple intranet devices to share a limited number of public network addresses via address sharing. However, NAT breaks the end-to-end principle, making it impossible for the extranet to directly access intranet devices and creating obstacles for remote interconnection.

A variety of technical solutions have been developed to tackle remote interconnection, but none can achieve perfect adaptation across all scenarios. The core features and limitations of mainstream solutions are analyzed as follows.

1.1 Server Relay

Server relay is the most widely adopted solution for remote interconnection. Leveraging the global reachability of public network servers, devices on different intranets establish indirect communication links through a shared relay server for data exchange. In practical deployment, developers can build custom relay services or utilize mature middleware, such as reverse proxy tools represented by frp and the MQTT framework. Reverse proxies create tunnel mappings between relay servers and intranet devices. When external users access a specified port on the relay server, traffic is forwarded to the target intranet device to realize indirect extranet access. The MQTT framework simplifies deployment and reduces development costs with ready-made components.

Nevertheless, server relay has obvious drawbacks. The detoured communication path increases network latency and brings a series of challenges in development, operation, maintenance and network security.

1.2 Port Mapping

Port mapping works by configuring rules on gateways to establish one-to-one mappings between public gateway ports and ports of intranet devices. When external users access a designated gateway port, data packets

are automatically forwarded to the corresponding port of the target intranet device, enabling extranet-to-intranet access.

There are fundamental differences between IP addresses and ports in functional positioning: an IP address is a network-layer identifier for locating terminal nodes in the network, while a port is a transport-layer identifier for distinguishing different application processes on a single terminal. Relying on port mapping for remote access causes functional dislocation between the network layer and the transport layer, resulting in management chaos. For instance, ports are generally owned by terminal users, whereas port mapping rules are configured by gateway administrators. The separation of rights and responsibilities easily leads to configuration conflicts. In addition, similar applications usually use the same default port across different nodes. Using ports to distinguish nodes goes against user habits and increases operational complexity. Accordingly, port mapping is only applicable to simple specific scenarios and cannot serve as a universal solution for remote interconnection.

1.3 VPN

VPN builds encrypted tunnels between different intranets to simulate a unified local area network environment, realizing interconnection of cross-regional intranet resources. It is widely applied in enterprise branch interconnection and remote office scenarios. This solution features high security and reuses existing intranet resources, but it has notable limitations in application scenarios. It requires all interconnected networks to belong to a unified administrative domain, such as the same enterprise or organization. Therefore, VPN is only a dedicated technology for specific scenarios and cannot act as a universal fundamental solution for remote interconnection.

1.4 IPv6

Since the root cause of remote interconnection issues lies in design defects of the IP protocol, the industry turned to the research and development of the next-generation IP protocol, namely IPv6 [5, 6, 7]. Adopting 128-bit long addresses, IPv6 effectively resolves the problem of address resource shortage. However, its flat address structure fundamentally conflicts with the inherent hierarchical structure of the Internet, leading to two critical flaws. First, it suffers from poor backward compatibility. Seamless interworking with IPv4 is unachievable, and upgrading massive existing IPv4 devices and network facilities incurs extremely high costs, hindering large-scale deployment. Second, long addresses bring great difficulties in configuration, management and memorization compared with IPv4, making IPv6 user-unfriendly. These drawbacks also make it inapplicable in certain scenarios such as internal networks. For the above reasons, IPv6 is unlikely to completely replace IPv4 or become a unified next-generation IP protocol.

2 IP++ Scheme Based on Variable-Length Addresses

As analyzed above, a new solution is urgently required to address remote interconnection problems. Drawing lessons from IPv6, the new scheme must ensure excellent backward compatibility. On one hand, the massive legacy network facilities make smooth transition an inevitable requirement. On the other hand, the existing TCP/IP network system operates well and meets most practical demands. Hence, the new solution should be an extension of the current system rather than a complete overhaul.

Hierarchical variable-length identifiers are widely used in real life, including postal addresses, telephone numbers, file paths and domain names. This common practice is not coincidental but stems from inherent rationality. All the above objects feature hierarchical aggregation, for which multi-level variable-length identification is the most natural and efficient approach. Network nodes also present typical hierarchical aggregation characteristics, so the addressing of Internet nodes should adopt a multi-level variable-length structure as well. Fixed-length addresses were only a temporary choice in the early stage when the number of network nodes was small and the network structure was simple. Although the "public address + private address" mode of IPv4 reflects partial ideas of multi-level variable-length addressing, it fails to form a systematic architecture.

The core idea of IP++ is to fully introduce multi-level variable-length addresses into the network layer protocol and build an addressing system matching the hierarchical network topology, so as to fundamentally

solve remote interconnection problems [8]. It is worth noting that the industry has long explored variable-length addresses since the emergence of the TCP/IP protocol suite. Adopting a full variable-length address architecture has always been the ultimate goal of IP protocol designers, while all commercial IP protocols in history are compromises between theoretical ideals and engineering realities. Variable-length address schemes were already taken into consideration in the initial design of TCP/IP, yet they were not implemented due to the limited hardware performance at that time [9]. Internet pioneers including David D. Clark, Vinton Cerf, J. Noel Chiappa and Christian Huitema have devoted themselves to theoretical research and technical practice of variable-length addresses for decades [10, 11]. The standardization of IPv6 was a critical opportunity to adopt variable-length addresses, and intense debates on this topic took place in the industry [12, 13]. Unfortunately, after multiple rounds of negotiation and compromise, IPv6 only adopted variable-length prefixes partially instead of a full variable-length address architecture.

As large-scale deployment of IPv6 encounters practical bottlenecks, researchers including Christian Huitema [14], Huawei 2012 Labs [15], Mao Decao, Qian Hualin and Wang Tao have continuously proposed to adopt variable-length addresses to break the deadlock. IP++ provides a systematic and engineerable implementation scheme for variable-length address theories.

2.1 Division of UnitNets

Under the IP++ framework, the Internet is divided into multiple levels. Networks at different levels and subnets at the same level all adopt independent address spaces, which are defined as unitnets. Each unitnet owns an address space equivalent to that of IPv4.

The Internet presents a tree topology. The top-level network is the public network, defined as the root node of the tree. Sub-unitnets are connected hierarchically below the public network to form a complete tree structure.

Both unitnets and connected terminals have clear level attributes, which can be expressed in two ways: absolute level and relative level. The absolute level is prefixed with "/", represented by a non-negative integer. The public network is defined as level /0, followed by level /1, /2 and so on downwards. The relative level is prefixed with ".", represented by an integer. The current unitnet is level .0, its parent unitnet is level .-1, the grandparent unitnet is level .-2, and so on upwards.

2.2 Address Representation

An IP++ complete address is formed by concatenating addresses of all corresponding levels, which follows the same logic as hierarchical identifiers such as postal addresses and file paths. A complete IP++ address must be preceded by the starting level, for example, /1#192.168.1.10.

IP++ also supports relative addresses, which can be understood by analogy with relative paths in file systems. For example:

.0#192.168.1.10 refers to the address [192.168.1.10](192.168.1.10) within the current unit network. .-1#10.66.7.250 refers to the address [10.66.7.250](10.66.7.250) in the parent unit network of the current network.

2.3 Header Format Design

The IP++ protocol consists of a mandatory basic header and optional extension headers, balancing core data transmission functions and expandability.

The basic header carries core information for packet transmission, such as type of service and time-to-live. It adopts a variable-length design, with its actual length specified by the "header length" field. The calculation formula is: Basic header length $= 8 + 8 \times 2^n$ bytes For example, if the "header length" field is set to 0, the actual header length is 16 bytes.

IP++ addresses consist of multiple fields due to their hierarchical features. Taking the destination address /#1.1.1.2-1.1.1.28 as an example:

The "destination address type" field is set to 0 to indicate an absolute address.

The "destination address parameter" field records the starting level and address depth: the first 4 bits represent the starting level (0 in this case), and the last 4 bits represent the hierarchical depth (1 level in this case), corresponding to an 8-byte address.

The 8-byte destination address is stored at the beginning of the combined "destination/source address" field, followed by the source address data starting from the 9th byte. The total length of this combined field is determined by the "header length" field.

IPv4 implements differentiated functionality through the "Options" field, but due to constraints such as length limitations and complex parsing, the actual application rate is extremely low. IP++ replaces the "options" field with "extension headers" to carry extended functions such as route optimization, security authentication and traffic control. "Extension headers" are optional and can be added on demand. New types of extension headers can be defined flexibly for diverse service requirements without modifying the basic header structure. Detailed specifications of "extension headers" are documented in relevant technical manuals.

3 Compatibility Design

Superior backward compatibility is one of the core advantages of IP++. Networks with IP++ devices and pure IPv4 devices can operate normally in a mixed deployment scenario. Terminals can run IP++ sockets and IPv4 sockets simultaneously. As an extension of IPv4 rather than a disruptive replacement, IP++ maintains strong continuity in address format and protocol logic.

Public network and private networks can be divided into independent unitnets, and each unitnet follows rules similar to IPv4. IP++ addresses are a superset of IPv4 addresses (IPv4 addresses can be regarded as IP++ /0-level addresses), so legacy IPv4 networks can be upgraded to IP++ without re-addressing. New and old networks can seamlessly connect, The network upgrade can be implemented gradually by stages and regions, maximizing the value of existing devices and network investments. In addition, technical staff do not need to reconstruct their knowledge system, and learning costs are extremely low.

IP++ realizes backward compatibility mainly through three strategies: encapsulation, translation and dual stack.

3.1 Encapsulation

The encapsulation mechanism of IP++ is essentially different from traditional encapsulation. Since there is an inherent mapping relationship between IP++ and IPv4 addresses, encapsulation and decapsulation can be completed automatically without manual configuration.

Encapsulation and decapsulation always work in pairs within a single unitnet. Packets must be in the decapsulated state when crossing the boundary between different unitnets.

3.2 Translation

Translation refers to direct conversion between IP++ packets and IPv4 packets. The prerequisite for this translation is address length matching—specifically, when the source address and destination address are within the same unitnet, IP++ can utilize a short address format (such as .#x.x.x.x), which facilitates direct conversion. If the source and destination addresses belong to different unitnets with mismatched address lengths, translation needs to work with the NAT mechanism. Traditional NAT's core purpose is to address IPv4 address scarcity. Under IP++, addresses are no longer scarce, so standalone NAT is typically of little significance. In IP++, NAT is typically used with translation to achieve cross-unitnet protocol conversion. The working principle of NAT in IP++ is similar to the traditional version. When a packet enters the target terminal unitnet, the gateway router replaces the original source address with the gateway address of the target unitnet and records the address mapping. When the response packet is returned, the gateway address is converted back to the original source address to realize bidirectional communication.

Different from traditional NAT which only supports intranet-to-extranet access, NAT in IP++ supports two scenarios for packets entering a terminal unitnet: transmission from a lower-level unitnet to an upper-level unitnet, and transmission from an upper-level unitnet to a lower-level unitnet. The former is similar to traditional NAT, while the latter is a new application scenario. A large number of legacy IPv4 devices

and embedded terminals prefer short addresses, so the reverse NAT function has strong practical value for interconnection with existing devices.

In summary, translation requires short addresses, which is only feasible when communication parties are in the same unitnet. For cross-unitnet communication, translation must cooperate with NAT. The combination of NAT and translation enables smooth interconnection between IP++ networks and massive legacy IPv4 devices such as public servers and embedded devices.

3.3 Dual Stack

This dual-stack is merely a transitional engineering implementation. In principle, IP++ and IPv4 can be deployed in single-stack form, and the final form will inevitably be single-stack, with IPv4 functionality as an adjunct to IP++ functionality.

According to the IP++ specification, devices supporting IP++ must also support IPv4. Therefore, in the protocol stack of devices supporting IP++, IPv4 and IP++ processing programs exist side by side, which is known as dual stack. The two stacks are not completely independent, because IP++ addresses include IPv4 addresses, and data packets can be switched between the two stacks. For example, when an IPv4 packet is received by an IP++ application, the protocol stack will switch the packet to the IP++ stack before delivering it to the transport layer. The reverse switching is also supported for outbound packets, and the protocol stack will record relevant session states.

For client-side packets, the protocol stack can transmit data via either stack by default without switching. Manual configuration only supports switching from IPv4 to IP++, while switching from IP++ to IPv4 is generally unnecessary.

The dual-stack mechanism guarantees excellent compatibility during network upgrades. Legacy IPv4 applications can run normally on IP++ devices without modification. Developers only need to develop new applications based on IP++ sockets, and the protocol stack will automatically handle IPv4 compatibility issues.

Dual-stack technology is of great significance for ensuring compatibility with IP++. When the operating system of the device is upgraded to IP++, it's common for applications to not be able to be updated in time. It would be unacceptable if applications could no longer be used due to this issue. Applications that only support IPv4 in existing IP++ devices can still function properly. When developing new applications, there's no need to worry about compatibility with IPv4; the protocol stack will handle all the necessary aspects automatically.

4 Typical Application Scenarios

The long-term goal of IP++ is to serve as a universal network-layer infrastructure deployed on all network devices including routers, terminals and servers. In the initial stage of ecosystem construction, IP++ will be firstly applied to scenarios with strong demands for remote interconnection. Typical scenarios are as follows.

4.1 Internet of Things

The rapid development of IoT generates massive demands for remote interconnection, such as remote control of smart home devices, remote maintenance of industrial equipment and remote uploading of environmental monitoring data. The end-to-end communication capability of IP++ fully meets these requirements, greatly reducing the development, deployment and operation costs of IoT applications and providing an efficient and reliable underlying network support for the IoT ecosystem.

4.2 Address Resource Expansion

The scarcity of public IPv4 addresses restricts the deployment of many network applications. The multi-level variable-length address architecture of IP++ provides nearly unlimited address resources. Telecom operators no longer need to worry about address allocation, and enterprises can easily deploy globally accessible terminals at low cost.

4.3 Data Center Networks

Data center networks feature an obvious hierarchical topology, which is highly matched with the multi-level variable-length address structure of IP++. Deploying IP++ simplifies network configuration and management, and reduces construction and operation difficulties.

4.4 Multipath TCP (MPTCP)

MPTCP [16] improves communication reliability and bandwidth utilization via multi-path transmission. However, the flat address structure of traditional IP protocols cannot carry hierarchical network information. As a result, the multi-path capability of MPTCP is limited to multi-interface binding, failing to optimize paths based on network hierarchy. The hierarchical address of IP++ can reflect network topology details and support refined path control. With IP++, MPTCP can fully utilize multi-path resources based on network hierarchy, giving full play to its technical advantages and expanding application scope.

5 Conclusion

The innovative multi-level variable-length address architecture of IP++ remedies the core defects of traditional IP protocols. It solves the problems of IPv4 address shortage and inconvenient remote interconnection, and avoids the shortcomings of IPv6 including poor compatibility and poor usability. IP++ restores the IP protocol to the end-to-end principle and acts as the complete version of the IP protocol.

As a fundamental network-layer technology, IP++ involves multiple aspects of network systems. This paper mainly elaborates its core ideas and key mechanisms, more technical details are presented in relevant official documents. Although IP++ still faces challenges in ecosystem building and standardization, the evolution of IP protocols towards multi-level variable-length addresses is an irresistible trend based on technical logic and practical demands. With the gradual improvement of the IP++ ecosystem, it is expected to become the unified next-generation IP protocol and drive the Internet towards a more complete, convenient and scalable future.

References

[1] X R Xie. *Computer Networks*. Publishing House of Electronics Industry, Beijing, 8th edition, 2021.

[2] K R Fall, W R Stevens, and G R Wright. *TCP/IP Illustrated*. China Machine Press, Beijing, 2016. Translated by Wu Ying.

[3] J. Postel. RFC 791: Internet protocol. IETF, 1981.

[4] P Srisuresh and M Holdrege. RFC 2663: Ip network address translator (nat) terminology and considerations. IETF, 1999.

[5] S Deering and R Hinden. RFC 8200: Internet protocol, version 6 (ipv6) specification. IETF, 2017.

[6] R Hinden and S Deering. RFC 4291: Ip version 6 addressing architecture. IETF, 2006.

[7] Y Rekhter, T Li, and R Hinden. RFC 4213: Basic transition mechanisms for ipv6 hosts and routers. IETF, 2005.

[8] Ltd. Beijing IpPlusPlus Network Technology Co. Ip++ protocol official website. http://www.ippp.xyz.

[9] D D Clark. *Designing an Internet (Information Policy)*. MIT Press, Cambridge, 2018.

[10] D D Clark, L Chapin, V G Cerf, et al. Towards the future internet architecture. Technical report, Internet Architecture Board, USA, 1991.

[11] J N Chiappa. Why topologically sensitive names are inherently variable length. Technical report, Massachusetts Institute of Technology, Cambridge, 1994.

[12] P Francis. RFC 1621: Pip near-term architecture. IETF, 1994.

[13] P Francis. RFC 1622: Pip header processing. IETF, 1994.

[14] C Huitema. Flexible ip: An adaptable ip address structure. Technical report, IETF 109, 2021.

[15] Z Chen, C Wang, G W Li, et al. NEW IP framework and protocol for future applications. In *2020 IEEE/IFIP Network Operations and Management Symposium (NOMS)*, pages 1–6. IEEE, 2020.

[16] A Ford, C Raiciu, M Handley, et al. RFC 6824: Tcp extensions for multipath operation with multiple addresses (mptcp). IETF, 2013.